Correzione: cicli, consegne e il menu delle figure

Lezione 14 — 1IOT, SST-ICT-01 — Domande sui compiti, regole delle consegne, Esercizio 6 del Compito C01

26/10/2026

1 Che cosa abbiamo fatto oggi

Oggi non abbiamo iniziato un argomento nuovo: abbiamo fatto una lezione di **correzione**. All'inizio ho scritto alla lavagna la **lista delle richieste** arrivate dai compiti per casa e dalle vostre domande:

- l'**indentatura** (Sezione 2);
- il **while** (Sezione 3);
- i **puntini** al posto dello spazio (Sezione 4);
- il **do while** (Sezione 5);
- un **resoconto sugli errori** degli esercizi (Sezioni 6 e 7).

Poi in aula avete svolto il **controllo** del compito per casa e nella seconda parte abbiamo **corretto insieme** l'**Esercizio 6** del Compito C01, il **menu delle figure** (Sezione 8), il più difficile della serie. La funzione e i suoi parametri, previsti per oggi, li vediamo nelle prossime lezioni.

2 L'indentatura

L'**indentatura** (o indentazione) è il modo di scrivere i comandi **con dei rientri**, cioè non tutti allineati al bordo sinistro: per esempio, dopo `if (x == 3)` e la graffa aperta, il `printf` non lo scrivo sotto la graffa, ma **spostato a destra**, lasciando uno spazio vuoto tra il bordo e l'inizio del testo.

2.1 In C non cambia il risultato

In C l'indentatura **non ha nessun significato esecutivo**: potrei scrivere tutto il programma su **un'unica riga** e funzionerebbe lo stesso. In altri linguaggi non è così: in **Python** (un altro linguaggio di programmazione, che si studia per esempio in alcuni licei) le graffe non ci sono, e l'indentatura serve proprio alla macchina per capire che un comando **dipende** da un'istruzione: tutti i comandi indentati sotto un `if` vengono raccolti dentro quell'`if`.

In C questo lavoro lo fanno le **graffe**. Guardiamo l'esempio della lavagna:

```c
int main()
{
    int x;
    x = 3;
    if (x == 3)
    {
        printf("Inserisci un numero: ");
        scanf("%d", &n);
        printf("...");
    }
    printf("...");
}
```

I tre comandi dentro le graffe (`printf`, `scanf`, `printf`) **dipendono tutti e tre** dal fatto che `x` sia uguale a 3: se `x` vale 3 li esegue tutti e tre, se `x` vale 7 li **salta**. Il `printf` dopo la graffa chiusa invece si esegue sempre. Il **contenitore** fatto dalle graffe serve a delimitare una porzione di codice e a trattarla **come se fosse un'unica istruzione**.

Definizione

L'**indentatura** sposta a destra di 3–4 caratteri le istruzioni che stanno **dentro** un blocco di graffe, per far vedere subito che quel gruppo di istruzioni è **un gruppo per conto suo**. In C serve solo a **chi legge** il codice, non alla macchina.

Nota

Perché la riga dopo l'`if` parte già spostata? Perché l'`if` di solito sta **già dentro** le graffe del `main`: ha già un rientro davanti, e quello che sta dentro l'`if` ne prende **un altro**. Ogni livello di graffe aggiunge un rientro.

2.2 Il tasto Tab

Di solito per indentare si usa la **tabulazione**, cioè il tasto **Tab** (quello con le due frecce, sopra il blocco maiuscole, a sinistra). In automatico sposta il testo di alcuni spazi e lo tratta **come un unico carattere**: se fai una tabulazione e poi torni indietro, il cursore torna indietro di un colpo intero, non si ferma a metà.

Quanto è larga la tabulazione dipende dall'**editor**: il Blocco note di Windows la mostra larga 8 caratteri (in aula avevo detto 5–6), altri editor 2, 3 o 4. Su onlinegdb dovrebbero essere 4. A noi non cambia niente: l'importante è usarla.

2.3 Il pulsante Beautify

Quando andate a capo dentro un blocco, onlinegdb mette già da solo il cursore **nella posizione giusta**. In più, onlinegdb (e altri programmi che useremo) ha un pulsante che fa l'indentazione corretta **in automatico**: si chiama **Beautify** ed è in alto, con il simbolo di due graffe `{}`.

Attenzione

Beautify **non sa** che cosa avevate in testa: indenta seguendo le regole, in base a **quello che è scritto**. Se avete sbagliato una graffa, indenta male anche lui, e non è colpa sua. Proprio per questo è utile: se dopo Beautify un `printf` che doveva stare fuori dall'`if` finisce **spostato dentro**, oppure la graffa finale resta sola, avete trovato l'**errore**.

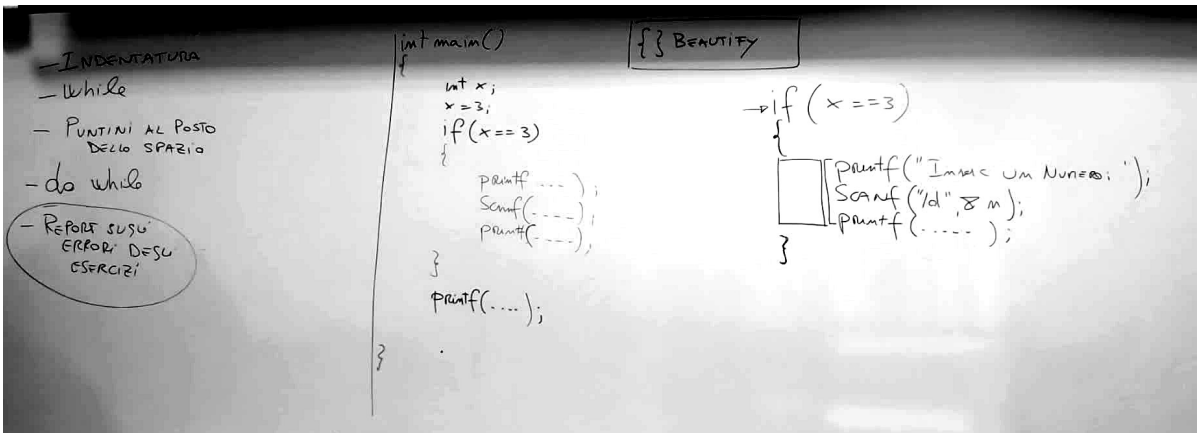


Figura 1: La lavagna di inizio lezione: la lista delle richieste, l'esempio con l'`if` indentato dentro il `main` e il pulsante Beautify. A destra il blocco di tre istruzioni che dipende dall'`if`.

2.4 Il trucco: prima la struttura

Per non dimenticare le graffe c'è un trucco che vale sempre: prima di scrivere il codice, **scrivete la struttura**.

1. Scrivo `if`, aperta tonda, chiusa tonda.
2. Vado a capo, **aperta graffa**, **chiusa graffa**.
3. Solo dopo entro tra le parentesi e **riempio gli spazi vuoti**: la condizione, i `printf`, gli `scanf`.

Se invece scrivo `if (x == 3)`, apro la graffa e poi `printf`, `scanf` e quant'altro, è facile **dimenticarsi la graffa** di chiusura per strada.

2.5 Perché farlo, se il programma funziona lo stesso?

Uno può dire: se il programma funziona, perché devo preoccuparmi dell'indentatura? Lo capisco io, gli altri si arrangino. Non funziona così:

- quello del programmatore **non è un lavoro solista**: si lavora in una **squadra**, e il codice lo legge e lo modifica anche altra gente;
- anche chi lavora da solo prima o poi deve **riprendere in mano** un codice scritto **tre anni prima**: non è possibile ricordarsi tutto.

Per questo l'indentatura è **fondamentale**, anche se non cambia il risultato.

Esempio

Domanda da verifica. L'indentatura pregiudica il risultato del programma?
Risposta: no. Pregiudica solo la **leggibilità**, cioè l'organizzazione grafica del codice.

3 Il while: che cosa fa davvero

Alla domanda «che cosa fa il while?» in aula sono arrivate risposte come «ripete finché non viene giusto» o «ripete la domanda finché non si è inserito il dato giusto». Sono risposte legate all'**esempio** che stavamo facendo (il controllo del dato), non a **che cosa è** il while.

Definizione

Il **while** è un **costrutto**, cioè una struttura della programmazione, che permette di **ripetere** una certa porzione di codice **finché una condizione è vera**. Non «finché c'è una condizione»: finché la condizione è **vera**. Quando diventa **falsa**, smette.

3.1 L'esempio dell'armadio

Programmo una macchina, un assistente, e gli dico: «**cammina** finché non arrivi all'armadio». Lui fa un passo e si chiede: sono arrivato all'armadio? No. Allora ripete. Un altro passo: sono arrivato? No. E avanti così. Quanti passi farà? Non si sa: ma **a ogni passo controlla** se è arrivato. Quando la risposta cambia, **smette** di camminare.

Attenzione

Per essere precisi, la domanda giusta è **al contrario**: «**non** sono ancora arrivato all'armadio?». Finché la risposta è **vera** (sì, è vero che non sono arrivato), faccio un altro passo. Quando sono arrivato, «non sono arrivato» diventa **falsa**, e mi fermo. Nel while è sempre la **condizione vera** che fa ripetere.

3.2 while e for sono la stessa cosa

```c
int n = 0;
while (n < 10)
{
    // ... istruzioni ...
    n++;
}
```

```c
for (int n = 0; n < 10; n++)
{
    // ... istruzioni ...
}
```

Figura 2: Lo stesso ciclo scritto con il while (a sinistra) e con il for (a destra), come alla lavagna.

Nel **while**, tra le tonde, va la **condizione di ripetizione**: `n < 10`, `n != 7`, qualunque condizione, dipende da quello che stiamo facendo. Dentro il blocco ci va il **modificatore**: `n++`, `n = n * 2`, `n = n + 2`, `n = n * 3 - 4 + 18 + k`, quello che volete. L'importante è che **n cambi**, altrimenti la condizione resta vera per sempre.

Il **for** fa la stessa cosa: fintanto che la condizione che si mette **nel mezzo** è vera, ripete. La differenza è solo di scrittura: nel for **inizializzazione**, **condizione** e **modificatore** stanno tutti e tre tra le tonde.

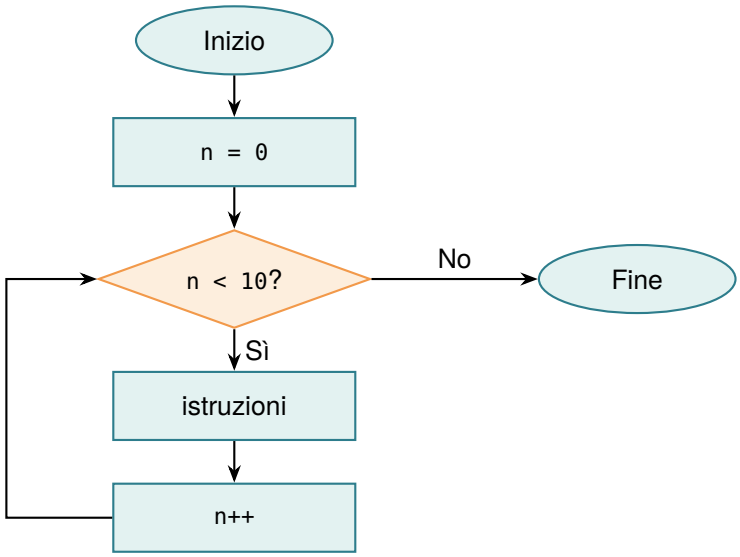


Figura 3: Diagramma di flusso comune ai due cicli: la condizione si controlla *prima* di ogni ripetizione.

3.3 Quando il for e quando il while

È una questione di **stile**. Di solito:

- si usa il **for** quando si sa **in anticipo** quante volte ripetere: un allenamento che prevede ogni volta **dieci salti**; lo so già che sono dieci;
- si usa il **while** quando **non si sa** quante volte ripetere: saltare **finché non si è stremati** dipende da quanto si resiste.

È solo una preferenza: niente vieta di usare il while con un numero fissato o il for senza sapere quante volte ripetere, se non le **regole non scritte** della programmazione, che tutti seguono, un po' come le regole di buona condotta.

4 I puntini al posto dello spazio

Negli esercizi con le figure, in certe posizioni si stampa l'asterisco, `printf("*");`, in altre uno **spazio**, `printf(" ");`. Nel testo degli esercizi, al posto degli spazi, c'erano dei **puntini**. Ed era scritto nel testo il perché: lo spazio **non si vede**, e si fa fatica a capire se in un punto c'è uno spazio oppure no. Il puntino serve solo a **occupare il posto dello spazio** per renderlo visibile.

Nel programma, dove nel disegno c'è un puntino, si stampa **lo spazio**. Se però avete stampato i puntini, non è un problema: l'obiettivo dell'esercizio è **esercitarsi sui cicli** di ripetizione, sugli `if`, sull'**annidamento**. Al posto dell'asterisco potete mettere il cancelletto, la chiocciola o quello che volete: l'importante è che i cicli siano fatti bene.

5 do while e while: l'unica differenza

Il **do while** ripete il blocco tra le graffe finché la condizione, scritta **in fondo**, è vera. Dentro il blocco ci può essere **qualunque cosa**: negli esercizi c'erano quasi sempre un `printf` e uno `scanf`, ma non vuol dire che ci sarà sempre solo quello. Fin qui è uguale al while. C'è **un'unica differenza**: il while può **non fare niente** già al primo colpo, il do while no.

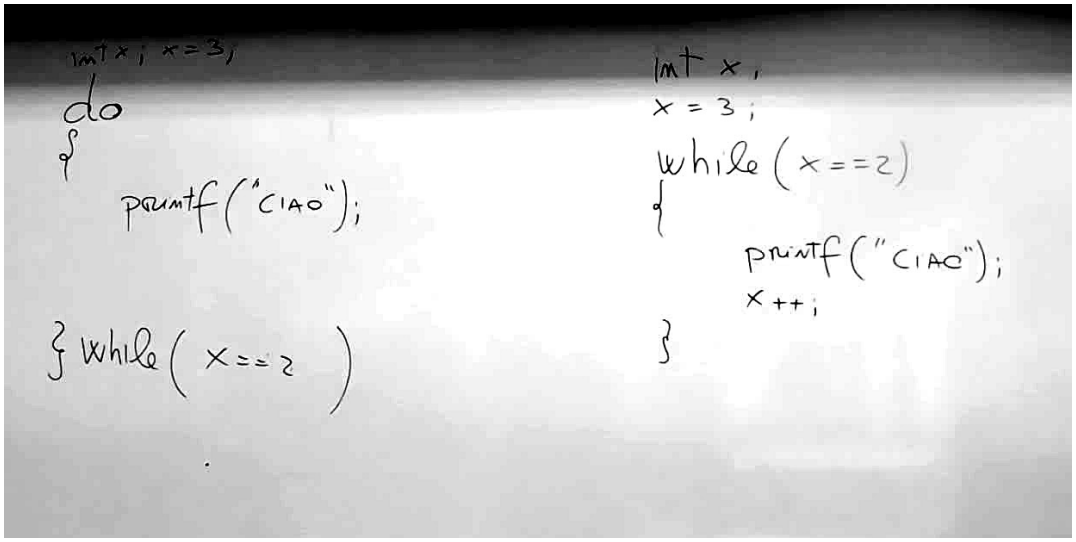


Figura 4: L'esempio della lavagna: a sinistra il do while, a destra il while, con la stessa condizione `x == 2` e `x` che vale 3.

```c
int x;
x = 3;
do
{
    printf("ciao");
} while (x == 2);
```

```c
int x;
x = 3;
while (x == 2)
{
    printf("ciao");
    x++;
}
```

Esempio

Quante volte stampa «ciao»?

- **while** (a destra): `x` vale 3 e il while dice «ripeti finché `x` è uguale a 2». La condizione è **già falsa** all'inizio: salta subito in fondo e **non stampa niente**. **Zero volte.**
- **do while** (a sinistra): la condizione è la stessa, sempre falsa, ma per arrivare a controllarla il programma deve **passare dal blocco**: quindi un «ciao» lo stampa. **Una volta.**

Il do while è come scrivere **una volta** il `printf("ciao")` e poi fare il while.

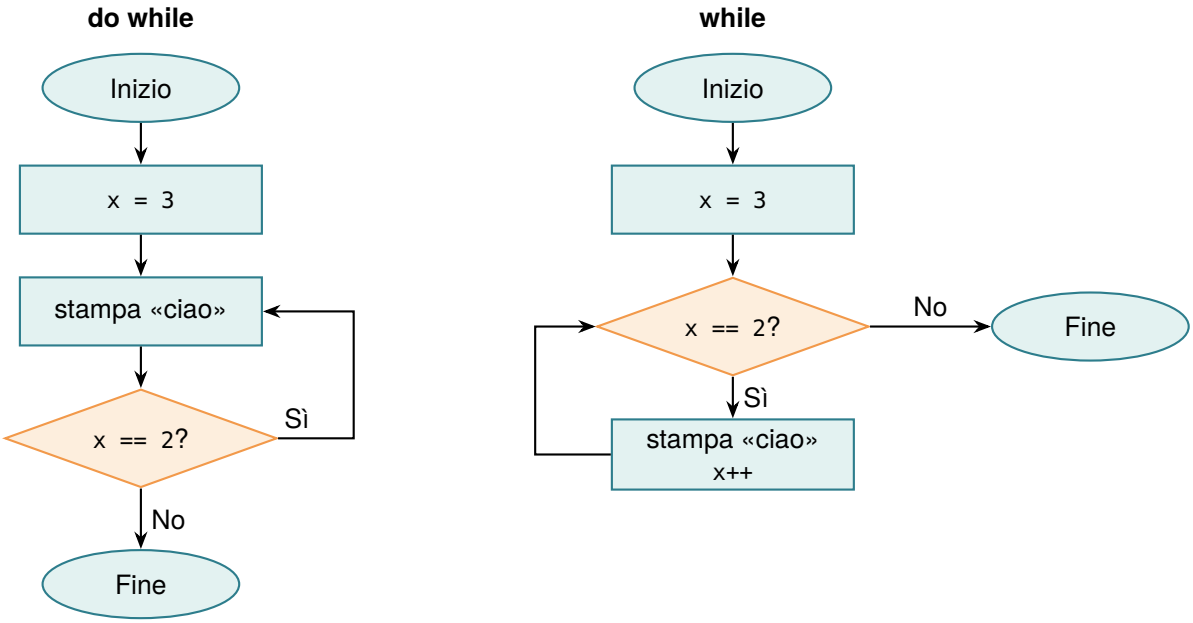


Figura 5: Nel do while la stampa viene *prima* della domanda, nel while *dopo*: con `x = 3` il primo stampa una volta, il secondo mai.

Nota

Quando si usa il do while? Se il blocco va eseguito **almeno una volta**, conviene il **do while** (per esempio una domanda all'utente, che va fatta almeno una volta). Con il while si può ottenere lo stesso risultato, ma bisogna preparare le variabili in maniera un po' «furba» prima del ciclo. Se non si sa se il blocco va eseguito, si usa il **while**.

Nota

Polling. In aula ogni tanto chiedo «*polling?*»: è il nome di una **tecnica** che consiste nel **chiedere a intervalli** se c'è qualcosa di nuovo; se nessuno risponde, vuol dire che non c'è niente. È quello che fate con il telefono quando lo guardate ogni minuto per vedere se è arrivato un messaggio. In classe vuol dire: «ci siete? ci sono domande?»

6 Le consegne dei compiti per casa

I compiti per casa vengono **valutati**, e il controllo in classe serve a capire se li avete **fatti voi**. Se dal controllo risulta che un compito consegnato tutto giusto non è stato fatto da chi l'ha consegnato, la valutazione **scende** fortemente. Correggere i propri errori va bene: **correggere** vuol dire sistemare quello che si è sbagliato. Consegnare un lavoro che non si sa spiegare è un'altra cosa.

6.1 La scadenza è una scadenza

La consegna ha un **orario**: se la scadenza è alle 20:00, l'ultimo istante utile è **19:59:59**. Alle 20:02 è **in ritardo**.

Esempio

Il bando pubblico. Ogni tanto lo Stato, la Regione o il Comune pubblicano un **bando**: una richiesta pubblica in cui dicono «cerchiamo qualcuno che ci faccia questo lavoro, mandateci un preventivo»; chi fa il preventivo migliore prende il lavoro. Se lavori per un'azienda e il bando è interessante, è un modo per guadagnare. Ma se la domanda arriva **anche un minuto dopo** la scadenza, **non viene nemmeno considerata**: il lavoro è perso. Le consegne vanno **rispettate**, sempre.

6.2 Il prossimo compito

Il nuovo pacchetto di esercizi (3–5 esercizi) arriva su Classroom e si consegna **entro sabato**, così la domenica resta il tempo per correggerli. Se avete un problema (niente computer, niente connessione), **non è una scusa**: si consegna prima della scadenza, oppure ci si fa prestare un computer da qualcuno, a casa di un amico, e si manda il lavoro per mail entro sabato sera. Non importa come: importa che **arrivi**.

Attenzione

I compiti per casa **sono obbligatori**. Chi dice il contrario dice una cosa **falsa**.

7 Disegnare l'output su una griglia

Nel controllo in classe ciascuno aveva un esercizio **diverso** (un rettangolo, un quadrato, un triangolo…) e una domanda come questa:

Esempio

«Se l'utente scrive 3 come lato, che cosa stampa il programma? Disegnalo qui sotto: **un carattere per quadretto**, **lascia vuoti gli spazi**.»

Molti disegni avevano la forma giusta, ma nel **posto sbagliato**: per esempio un rettangolo 3×3 disegnato al **centro** della griglia, con quattro quadretti vuoti prima.

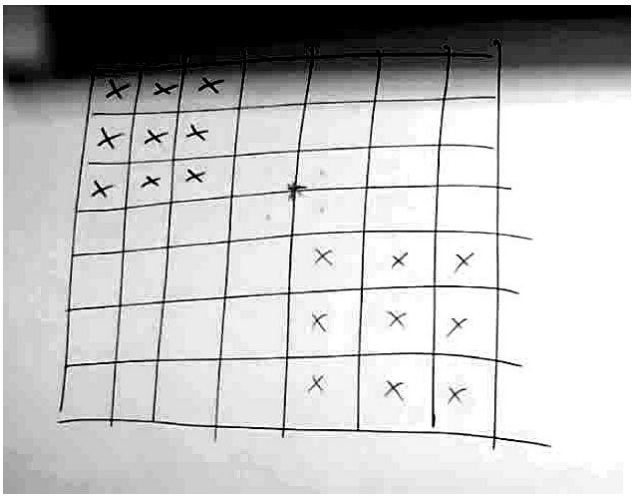


Figura 6: Alla lavagna: il 3×3 giusto, in alto a sinistra, e lo stesso 3×3 spostato al centro della griglia, che il programma non stampa mai.

Il computer **non sposta** il disegno perché «in angolo è brutto»: stampa **dall'inizio della riga**, cioè in **alto a sinistra**, e va avanti solo con gli spazi e gli a capo che gli abbiamo detto di stampare. Bisogna imparare a ragionare **come il computer**, che **non interpreta**: esegue quello che gli diciamo di fare.

Le regole per disegnare l'output:

1. si parte dal quadretto in **alto a sinistra**;
2. **un carattere per quadretto**, scritto **dentro** il quadretto e **non sull'incrocio** delle righe, altrimenti non si capisce in quale dei quattro quadretti intorno volevate metterlo;
3. dove il programma stampa uno spazio, il quadretto resta **vuoto**: niente puntini, se la consegna dice «lascia vuoti gli spazi».

Nota

Il panino. Chiedo un panino con le melanzane e mi danno un panino con un'altra cosa: perché? Se le melanzane non ci sono, si chiede, non si cambia la richiesta di testa propria. Con le consegne è lo stesso: si fa **quello che c'è scritto**, e se c'è un dubbio **si chiede**. Questi errori non sono una punizione: servono a imparare a **stare attenti ai dettagli**, perché la forma giusta nel posto sbagliato **penalizza** comunque la valutazione.

8 Esercizio 6 del C01: il menu delle figure

Quando c'è tempo la correzione dei compiti si fa **in aula**, come oggi; altrimenti vi arriva solo il **file con le correzioni**. Nella seconda parte della lezione abbiamo corretto insieme su onlinegdb l'**Esercizio 6** del Compito C01 (nel documento del compito è il capitolo 7), il più lungo della serie. Il testo, copiato come commento in testa al programma:

Esempio

Testo. Scrivi un programma con un **menu** che si ripete finché l'utente **non sceglie 0**: 1 = quadrato, 2 = quadrato vuoto, 3 = triangolo, 0 = esci.

- Con 1, 2 o 3 chiede il **lato** (da 1 a 9, **controllato**) e disegna la figura: quadrato pieno, quadrato vuoto (solo il bordo), triangolo rettangolo;
- con un numero che non è nel menu stampa **Scelta non valida**;
- il programma **conta** quante figure ha disegnato; quando si sceglie 0 stampa `Figure disegnate:` e il numero.

Suggerimento: uno **switch** dentro un **do while**, come il menu della Lezione 12. Il contatore parte da 0 prima del ciclo e aumenta di 1 (`figure++`) ogni volta che si disegna una figura. Puoi scrivere i tre disegni in tre `case` separati, copiando i programmi che hai già fatto.

8.1 Il ciclo del menu: do while

Il menu si deve **ripetere**, e la domanda va fatta **almeno una volta**: serve un **do while**. Si esce solo quando l'utente sceglie 0, quindi si ripete finché la scelta è **diversa da 0**: `while (scelta != 0)`.

Dentro il ciclo si chiede la scelta con un `printf`, in cui conviene scrivere anche le **opzioni**, così l'utente sa che cosa può scegliere, e la si acquisisce con lo `scanf`.

```c
int scelta;

do
{
    printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
    scanf("%d", &scelta);

    // ... qui le figure ...

} while (scelta != 0);
```

8.2 Le scelte: switch

Il testo suggerisce lo **switch**, «come il menu della Lezione 12». Davanti a un suggerimento così, la prima cosa da fare è **aprire la dispensa A12** e cercare l'esempio del menu: lì c'è esattamente la struttura che serve. Si potrebbe fare anche con gli `if`, ma lo switch è più **elegante** per le **scelte multiple**, perché le compatta.

Lo switch si fa sulla variabile in cui abbiamo salvato la scelta, cioè `scelta`. I casi sono:

- `case 1:`, `case 2:`, `case 3:` — le tre figure;
- `case 0:` — la **chiusura** (si stampa il numero di figure disegnate);
- `default:` — **tutti gli altri** numeri: «Scelta non valida».

I **due punti** dopo `case 1` fanno parte della scrittura del costrutto, come il `break` alla fine di ogni caso.

8.3 Il lato: dove chiederlo?

Per 1, 2 e 3 bisogna chiedere il lato e **controllarlo** (da 1 a 9). Ci sono tre possibilità:

1. scrivere `printf`, `scanf` e il do while di controllo **tre volte**, dentro ogni `case`: da evitare. Se scrivo tre volte la stessa cosa, **rischio di sbagliare tre volte**, e se devo fare una modifica devo farla **tre volte**;
2. mettere **prima dello switch** un `if` che chiede il lato solo se la scelta vale 1, 2 o 3: è la soluzione **preferita**;
3. mettere `case 1:`, `case 2:` e `case 3:` **in sequenza**, senza `break`: si può fare, ma poi per distinguere le figure servirebbe comunque un `if` sotto, e il codice viene meno pulito.

Con la seconda soluzione: se la scelta è tra 1 e 3 si chiede il lato; se è 0 (esci) o un numero non valido, il lato non serve.

```c
if (scelta >= 1 && scelta <= 3)
{
    do
    {
        printf("Inserisci il lato (da 1 a 9): ");
        scanf("%d", &lato);
    } while (lato < 1 || lato > 9);
}
```

Attenzione

In aula, come primo tentativo, per il do while è venuta in mente la condizione sulla scelta (`scelta >= 1 && scelta <= 3`): è sbagliata, perché quella l'abbiamo già usata nell'`if`. La condizione del do while riguarda il **lato**, non la scelta: si ripete la domanda **se il lato è sbagliato**, cioè `lato < 1 || lato > 9`. Se chiedo un numero da 1 a 9 e l'utente scrive 18, la domanda si ripete.

Si può risolvere l'esercizio anche in altri modi, e non è sbagliato: ma la serie di esercizi era costruita su **switch, for e while**, e l'esercizio va risolto con quelli. Se adesso sembra complicato: fra tre mesi, guardandovi indietro, vi sembrerà facile.

8.4 Caso 1: il quadrato pieno

Per un disegno in **due dimensioni** servono **due for annidati**: quello esterno sulle righe (`y`), quello interno sulle colonne (`x`). Per il quadrato tutti e due vanno fino a `lato`, perché i lati sono uguali.

```
case 1:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

Il for su `x` produce **una riga**; questa riga si ripete per `y` che vale 0, 1, 2, 3, 4 (con lato 5): si ferma a 4 perché parte da 0. Il `printf("*")` è l'unica istruzione del for interno: le graffe non servirebbero, ma **le mettiamo sempre**. Dopo la tonda chiusa del `for(...)` **non si mette** il punto e virgola: un `;` lì vorrebbe dire «ripeti un'istruzione vuota», e il blocco tra le graffe verrebbe eseguito una volta sola.

Nota

Provare a pezzi. Scritto solo il `case 1`, si prova il programma scegliendo **solo 1**: il 2 e il 3 non sono ancora gestiti, e con un'altra scelta il programma non fa niente (o fa cose strane). E come sempre: se allo `scanf` si scrive un carattere non corretto, per esempio una **lettera** al posto del numero, il programma **impazzisce**.

Attenzione

Alla **fine di ogni riga**, cioè dopo il for su `x` e dentro il for su `y`, ci vuole `printf("\n")` per **andare a capo**: senza, tutti gli asterischi finiscono **in fila** sulla stessa riga.

8.5 Il primo errore: `lato` non dichiarata

Alla prima prova il programma **non si compilava**. In aula avevo dichiarato `int lato;` **dentro le graffe dell'if**, subito prima del do while:



Figura 7: onlinegdb: `int lato;` è alla riga 29, dentro le graffe dell'`if`. Alla riga 40 il compilatore segnala `error: 'lato' undeclared (first use in this function)`.

Una variabile dichiarata **dentro un blocco di graffe** esiste **solo dentro quel blocco**: quando il blocco finisce, la variabile viene **distrutta**. Lo switch è fuori dall'`if`, quindi lì `lato` **non esiste** più, e il compilatore dice che non è dichiarata.

Definizione

Correzione: spostare `int lato;` **in cima** al `main`, insieme a `int scelta;`, fuori da ogni blocco. Così la variabile è visibile in tutto il `main`.

8.6 Quando il programma non si compila

In aula alcuni programmi non funzionavano anche dopo questa correzione. Le cose da fare, nell'ordine:

1. **Leggere il messaggio** di errore: per esempio onlinegdb diceva che **mancava una graffa**.
2. Premere **Beautify** e guardare l'indentazione: in un caso c'era una graffa **in più**, sotto un `for`, da cancellare, e poi mancava una graffa di chiusura **in fondo**, dopo l'ultima riga. Attenzione a leggere bene prima di correggere: in aula all'inizio avevo letto male, scambiando una graffa giusta per sbagliata, e ho dovuto annullare con `Ctrl+Z`. Con l'indentazione si vede subito quale blocco non è chiuso.
3. Controllare blocco per blocco **dove si chiude** ogni graffa: quella del `main`, quella del `do`, quella dello `switch`.
4. **Confrontare** il proprio codice, riga per riga, con il programma corretto (Sezione 8.10).

Nota

Un errore che non si vede. In un caso il compilatore segnalava un errore su una riga che sembrava **perfettamente corretta**, forse per un **carattere non riconosciuto**: togliendo quel pezzo di codice il programma funzionava, rimettendolo no. Può succedere quando nel codice finisce un carattere **strano o invisibile** (per esempio copiando del testo da un'altra parte). Il consiglio: **cancellare** quella parte e **riscriverla** a mano.

8.7 Caso 2: il quadrato vuoto

Il quadrato vuoto è **sempre un quadrato**: i due for annidati si **ricopiano** dal caso 1. La differenza è che a volte si stampa l'**asterisco** e a volte uno **spazio**.

Si stampa l'asterisco solo sul **bordo**, cioè in quattro situazioni (con lato 5, `x` e `y` vanno da 0 a 4):

- `x == 0`: prima **colonna**;
- `y == 0`: prima **riga**;
- `x == lato - 1`: ultima colonna (**lato meno 1**, perché non si arriva a 5 ma a 4);
- `y == lato - 1`: ultima riga.

In tutti gli altri casi si stampa uno **spazio**. Lo spazio **va stampato**, anche se non si vede: altrimenti gli asterischi del lato destro **scivolano a sinistra**, come era successo in uno degli esercizi della lezione scorsa.

```
case 2:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.8 Caso 3: il triangolo

Il triangolo l'abbiamo già fatto nella Lezione 13: è sempre un **doppio for annidato**, ma sulle `x` si arriva **fino a y** (`x <= y`): la prima riga ha un asterisco, la seconda due, e così via fino in fondo.

```
case 3:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.9 Il default e il contatore delle figure

Se la scelta non vale né 1, né 2, né 3, né 0, si finisce nel `default`, che stampa «Scelta non valida».

Il programma deve anche **contare** le figure disegnate: serve una variabile in più, `int nFigure = 0;`, che parte da **zero** **prima del ciclo**. In ognuno dei tre casi delle figure, **prima del break**, si aumenta di 1: `nFigure++;`. Quando l'utente sceglie 0, il `case 0` stampa il risultato:

```
case 0:
    printf("Le figure disegnate sono %d\n", nFigure);
    break;
default:
    printf("Scelta non valida\n");
```

8.10 Il programma completo

È il programma scritto in aula, con le correzioni del box che segue (provato con gcc).

```c
#include <stdio.h>

int main()
{
    int scelta;
    int lato;
    int nFigure = 0;

    do
    {
        printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
        scanf("%d", &scelta);

        // il lato serve solo per le tre figure
        if (scelta >= 1 && scelta <= 3)
        {
            do
            {
                printf("Inserisci il lato (da 1 a 9): ");
                scanf("%d", &lato);
            } while (lato < 1 || lato > 9);
        }

        switch (scelta)
        {
            case 1:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 2:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
                        {
                            printf("*");
                        }
                        else
                        {
                            printf(" ");
                        }
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 3:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x <= y; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 0:
                printf("Le figure disegnate sono %d\n", nFigure);
                break;
            default:
                printf("Scelta non valida\n");
        }

    } while (scelta != 0);

    return 0;
}
```

Attenzione

Due errori rimasti nel codice scritto in aula (che non abbiamo fatto in tempo a provare):

- nel `case 2` l'ultima condizione era `(y = lato - 1)`, con **un solo uguale**. Un solo = **assegna** il valore, non lo **confronta**: la condizione diventa sempre vera (con lato maggiore di 1) e in più cambia `y`. Con lato 5 il programma stampa **due righe piene** di asterischi invece del quadrato vuoto, e il compilatore **non dà nessun errore**. Per confrontare servono **due uguali**: `(y == lato - 1);`
- nel `case 2` mancava il **punto e virgola** dopo `nFigure++`: questo invece il compilatore lo segnala.

In più ho aggiunto `\n` in fondo ai messaggi del `case 0` e del `default`, così il menu successivo riparte su una riga nuova.

8.11 Una prova

```
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 1
Inserisci il lato (da 1 a 9): 12
Inserisci il lato (da 1 a 9): 4
****
****
****
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 2
Inserisci il lato (da 1 a 9): 5
*****
*   *
*   *
*   *
*****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 3
Inserisci il lato (da 1 a 9): 4
*
**
***
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 7
Scelta non valida
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 0
Le figure disegnate sono 3
```

8.12 Il diagramma di flusso

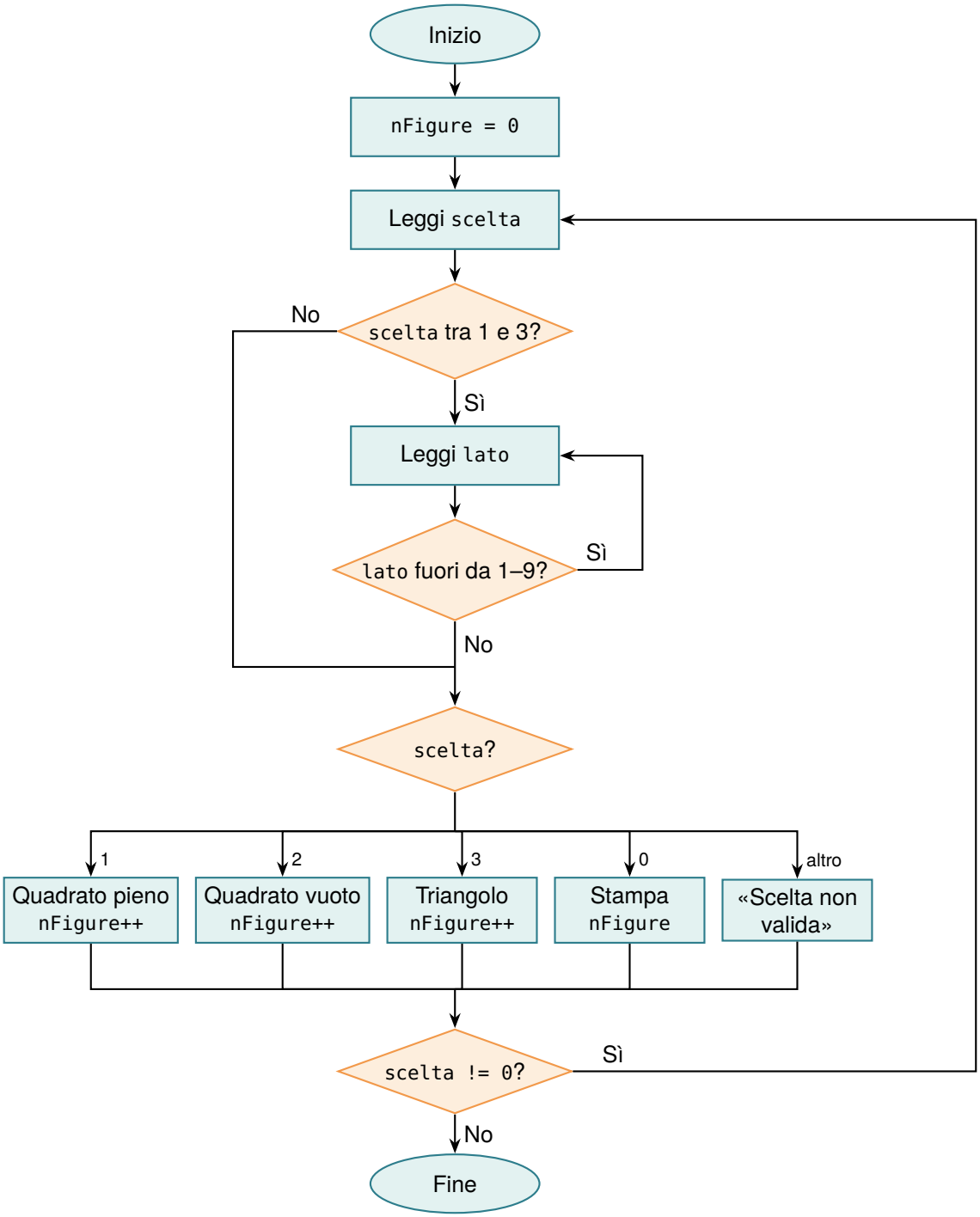


Figura 8: Il menu delle figure: lo `switch` (rombo «`scelta?`») sta dentro il do while, che si ripete finché la scelta è diversa da 0.

9 Cosa ricordare di questa lezione

- In C l'**indentatura** non cambia il risultato, ma rende il codice **leggibile**; il pulsante **Beautify** la sistema e aiuta a trovare le graffe sbagliate. Prima si scrive la **struttura** (parentesi e graffe), poi si riempie.
- Il **while** ripete un blocco **finché la condizione è vera**; il **for** fa la stessa cosa, con inizializza-

zione, condizione e modificatore tra le tonde. For quando si sa **quante volte**, while quando non si sa.

- Il **do while** esegue il blocco **almeno una volta**, anche se la condizione è falsa; il while può non eseguirlo mai.
- Le **scadenze** si rispettano al minuto; i compiti per casa sono **obbligatori**.
- L'output su una griglia si disegna partendo **in alto a sinistra**, **un carattere per quadretto**, spazi **vuoti**.
- Una variabile dichiarata **dentro un blocco** di graffe esiste **solo lì**: se serve anche fuori, va dichiarata in cima al `main`.
- Per confrontare servono **due uguali** (==); un solo uguale **assegna**, e il compilatore non se ne accorge.

Compito per casa. Su Classroom trovate il nuovo pacchetto di esercizi: la consegna è **entro sabato**. Prima di farlo, **confrontate** il vostro Esercizio 6 con il programma completo di questa dispensa e cercate dove stava l'errore.